

**Advanced JavaScript Backend Frameworks
(Node.js & Express JS)**

CIA III

Project Report on

STAYO

Hotel Booking and Property Management Platform

GitHub Link: <https://github.com/JuliusDude/stayo>

COMPLETED BY:

S.No	Student Name	Roll No.
1	Julius B Thomas	2462095
2	Kripa Maria Jestin	2462100
3	Kuragayala Rachel	2462106
4	Joshua Kuriakose Mathew	2462092

Course Name: Advanced JavaScript Backend Frameworks

Class: 5 BTCSAIML A

Semester: 5

ABSTRACT

Stayo is a comprehensive, production-grade hotel booking and property management backend platform. Engineered to eliminate operational inefficiencies in hospitality workflows, Stayo provides an integrated service layer where guests can securely search real-time room availability, reserve accommodations, and review itemized invoices, while hotel administrators and staff manage inventory, dynamic seasonal pricing rules, check-in workflows, and housekeeping lifecycles.

The application is architected as a RESTful API service on the Node.js runtime utilizing Express.js, with persistent storage managed via MongoDB and the Mongoose Object Data Modeling (ODM) framework. Application security is enforced using bcrypt password hashing, stateless JSON Web Token (JWT) authentication, and strict role-based access control (RBAC). The backend implements 13 functional modules, supporting advanced relational data modeling across collections, non-overlapping availability calculations, state machine workflows for reservations, and real-time revenue aggregation.

OBJECTIVE

The core objective of the Stayo backend platform is to implement a robust, maintainable, and scalable enterprise reservation infrastructure. The specific objectives include:

- **Secure Authentication & Authorization:** Implement salted bcrypt password encryption and stateless JWT generation, enforcing fine-grained guest, staff, and admin access controls.
- **Property & Inventory Hierarchy:** Model property catalogs with structured hotel profiles, room type specifications, and physical room tracking with individual capacities.
- **Date-Overlap Availability Computation:** Design mathematically rigorous database queries preventing double-booking across arbitrary check-in and check-out windows.
- **Dynamic Multi-Tier Pricing Engine:** Provide configurable multiplier rules based on seasons, weekends, and date ranges that automatically adjust base rates during reservation creation.
- **Enforced Booking State Machine:** Govern reservation progression through strict lifecycle phases (Reserved, Confirmed, Checked-in, Checked-out, Cancelled) rejecting illegal state transitions.
- **Operational Staff Workflows:** Enable hotel staff to execute verified check-in/check-out timestamp logging and maintain real-time housekeeping statuses.
- **Financial Management & Auditing:** Automatically compute invoices detailing stay costs, dynamic taxes, add-ons, and cancellation refunds, alongside admin occupancy and revenue analytics.

TECHNOLOGY USED

The system was developed strictly with modern, production-grade JavaScript backend technologies:

Category	Technology / Details
Backend Framework	Express.js (v4.x)
Runtime Environment	Node.js (LTS)
Programming Language	JavaScript (ECMAScript 2022+)
Database	MongoDB (NoSQL Document Store)
Object Data Modeling (ODM)	Mongoose
Authentication	JSON Web Tokens (JWT)
Password Security	bcrypt.js (10 Salt Rounds)
Request Validation	express-validator Middleware
API Architecture	RESTful JSON API
Error Handling	Centralized Express Middleware Pattern
API Testing & QA	Postman Collections (Member 1, Member 2, Member 3)
Environment Configuration	dotenv
Software Architecture	Model-View-Controller (MVC) Pattern

SCOPE OF THE PROJECT

The functional scope of Stayo spans across seven primary domains:

- **Authentication & Authorization:** Guest and staff registration with input validation; stateless JWT issuance; role-based authorization guarding guest, staff, and admin routes.
- **Hotel & Property Management:** Administrative creation, retrieval, and modification of hotel profiles; cataloging amenities, star ratings, and geographical locations.
- **Room & Inventory Management:** Hierarchical classification of room types (capacity, base price, room counts); individual physical room mapping and housekeeping status tracking.
- **Reservation Engine:** Conflict-free availability search based on guest counts and date boundaries; multi-night stay calculation with dynamic pricing multipliers.
- **Operations & Housekeeping:** Physical arrival (check-in) and departure (check-out) timestamp tracking; real-time room status toggle (Clean, Dirty, Maintenance); tiered cancellation policy with automated refund computations.
- **Guest Accounting:** Authenticated guest reservation history with data isolation; granular invoice generation detailing room rates, taxes, add-ons, and refunds.
- **Administration & Business Intelligence:** Creation and scheduling of dynamic pricing multiplier rules; system-wide occupancy rate computation and revenue reporting by hotel.

KEY FEATURES

Guest Authentication: Users securely create accounts and authenticate using their email and password. Passwords are hashed using bcrypt with a salt factor of 10. Upon successful authentication, a digitally signed JWT is issued containing user identity and role claims, allowing stateless verification across subsequent requests.

Hotel Management: Administrators can register new hotel properties within the platform. Each record captures comprehensive metadata including hotel name, city location, descriptive details, star ratings, and supported amenities (WiFi, Pool, Parking).

Room & Inventory Management: Stayo enforces strict referential integrity between properties, room categories, and physical rooms. Properties define `RoomTypes` which determine base pricing and maximum guest capacities. Physical `Rooms` reference specific room types and are assigned unique room numbers, ensuring inventory counts are always accurate.

Availability Search: The availability engine allows guests to query hotels by city, check-in date, check-out date, and number of guests. The backend executes an optimized query checking for overlapping reservations: a room is deemed unavailable if an existing active booking satisfies `(requestedCheckIn < existingCheckOut) AND (requestedCheckOut > existingCheckIn)`. Only unreserved rooms meeting capacity constraints are returned.

Reservation Workflow: When a reservation request is submitted, the booking controller re-verifies room availability to prevent race conditions. The total amount is computed dynamically across each night of the stay, integrating any active pricing rules. The reservation document is then committed with an initial status of `Reserved`.

Dynamic Pricing Engine: Administrators can define date-bound `PricingRules` tied to specific room types with assigned multipliers (e.g., 1.25x for peak holiday dates). During reservation calculation, the pricing calculator evaluates each individual night of the stay against active pricing rules, automatically applying applicable multipliers to produce an accurate total cost.

Booking Lifecycle Management: Bookings are governed by a strict finite state machine: `Reserved` → `Confirmed` → `Checked-in` → `Checked-out` or `Cancelled`. The controller explicitly validates the current status before permitting transitions, preventing invalid operations such as checking out an unconfirmed booking.

Check-in / Check-out Tracking: Hotel staff members process guest check-ins upon physical arrival, updating the booking status to `Checked-in` and writing an immutable timestamp (`actualCheckInAt`). When the guest departs, staff invoke the check-out endpoint, recording `actualCheckOutAt` and setting the room housekeeping status to `Dirty` for cleaning.

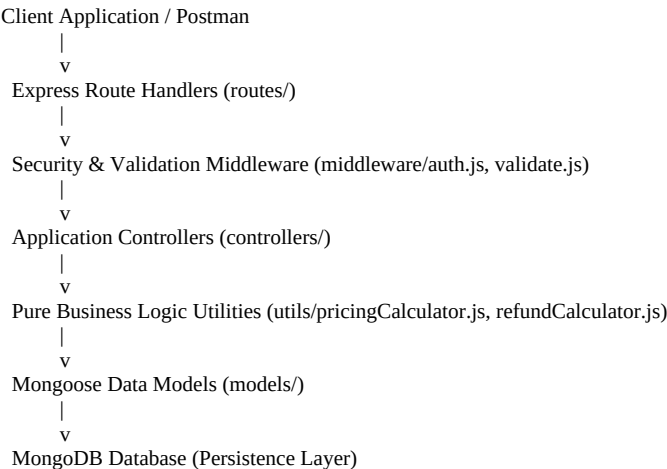
Housekeeping Status Tracking: Physical rooms maintain operational housekeeping statuses (`Clean`, `Dirty`, `Maintenance`). Staff can query rooms filtered by housekeeping status and update them once sanitation is completed, ensuring front-desk staff only assign clean rooms to arriving guests.

Cancellation & Refund Calculation: Guests or staff can cancel reservations subject to an automated tiered refund policy. The cancellation utility calculates the time delta between the cancellation request and scheduled check-in: cancellations made 7+ days in advance receive a 100% refund, 2-6 days receive a 50% refund, and less than 48 hours receive a 0% refund.

Guest Booking History & Invoices: Authenticated guests can retrieve their historical and active reservations (`guestId === req.user.userId`). A dedicated invoice endpoint aggregates base accommodation costs, dynamic taxes (defaulting to 10%), incidental add-ons, and applicable refunds, calculating a transparent final payable amount.

SYSTEM ARCHITECTURE

The Stayo backend strictly adheres to the Model-View-Controller (MVC) architectural pattern, separating routing, request interception, business domain logic, and data persistence:



Routing & Middleware: Maps HTTP requests to controllers, authenticates JWTs, and validates request bodies.

Controllers & Logic: Coordinates the request lifecycle, invokes pricing and refund logic, and formats JSON outputs.

Models & Persistence: Mongoose schemas enforce data types, default values, references, and indexes in MongoDB.

DATABASE DESIGN

Collection	Schema Fields & Constraints
Users	name (String), email (String, Unique, Indexed), passwordHash (String), role (Enum: 'guest', 'staff', 'admin')
Hotels	name (String), city (String, Indexed), amenities ([String]), rating (Number: 1-5, Default: 4)
RoomTypes	hotelId (ObjectId -> Hotel), name (String), basePrice (Number), capacity (Number), totalRooms (Number)
Rooms	roomTypeId (ObjectId -> RoomType), roomNumber (String, Unique), housekeepingStatus (Enum: 'clean', 'dirty', 'maintenance')
PricingRules	roomTypeId (ObjectId -> RoomType), multiplier (Number), startDate (Date), endDate (Date)
Bookings	guestId (ObjectId -> User), hotelId (ObjectId -> Hotel), roomId (ObjectId -> Room), roomTypeId (ObjectId -> RoomType), checkInDate (Date), checkOutDate (Date), status (Enum), totalAmount (Number), taxes (Number), addOns (Number), refundAmount (Number), actualCheckInAt (Date), actualCheckOutAt (Date)

SECURITY & VALIDATION

Stayo incorporates defense-in-depth principles: salted bcrypt password hashing, stateless signed JWT generation, strict role-based access control, resource ownership validation preventing horizontal data leakage, schema-level sanitization using `express-validator`, and centralized asynchronous error shielding preventing internal stack trace disclosure.

API ENDPOINTS

The Stayo platform provides 19 fully implemented, tested, and documented endpoints:

Method	Endpoint	Description & Access
POST	/api/auth/register	Register a new guest or staff user (Public)
POST	/api/auth/login	Authenticate user and return signed JWT (Public)
POST	/api/hotels	Create a new hotel profile (Admin)
GET	/api/hotels	List all registered hotels (Public)
GET	/api/hotels/search	Search availability by city, dates, and guests (Public)
POST	/api/room-types	Create room type under hotel (Admin)
POST	/api/rooms	Add a physical room to property inventory (Admin)
DELETE	/api/rooms/:id	Remove a physical room from inventory (Admin)
POST	/api/bookings	Reserve an available room (Authenticated Guest)
GET	/api/bookings/history	Retrieve personal booking history (Guest)
PUT	/api/bookings/:id/confirm	Confirm reservation status (Staff / Admin)
PUT	/api/bookings/:id/checkin	Process guest check-in with timestamp (Staff)
PUT	/api/bookings/:id/checkout	Process check-out and mark room dirty (Staff)
PUT	/api/bookings/:id/cancel	Cancel booking and compute tiered refund (Guest / Staff)
GET	/api/invoices/:bookingId	Generate itemized invoice summary (Owner / Staff)
POST	/api/pricing-rules	Configure dynamic seasonal pricing rule (Admin)
PUT	/api/rooms/:id/housekeeping	Update physical room housekeeping status (Staff)
GET	/api/reports/occupancy	Calculate property occupancy percentage (Admin)
GET	/api/reports/revenue	Aggregate total revenue by hotel property (Admin)

OUTPUT SCREENSHOTS

The application includes a clean operational interface and comprehensive API endpoints. Below are actual output captures of the running application:

Fig 1: Stayo Application Operational Overview

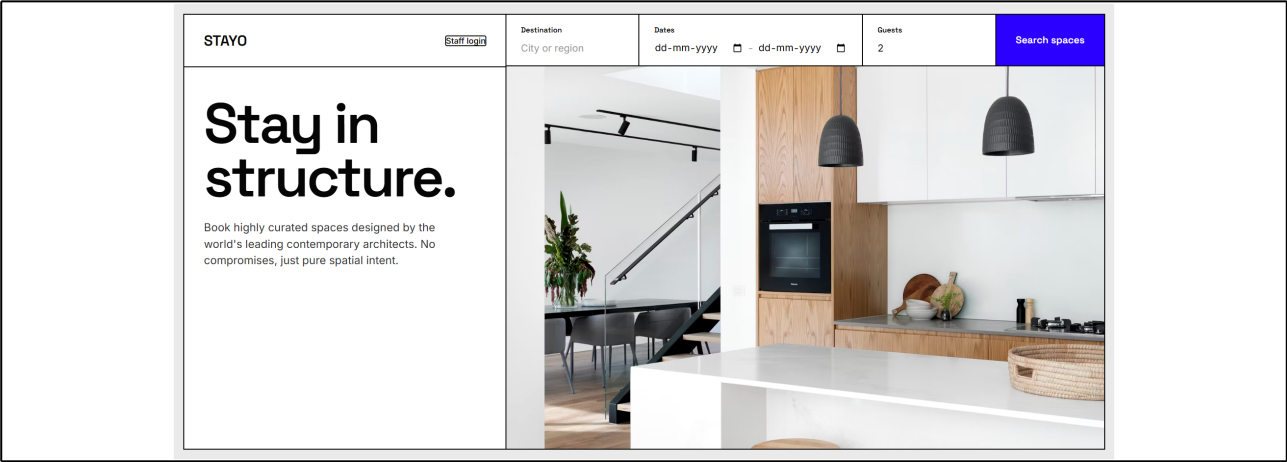
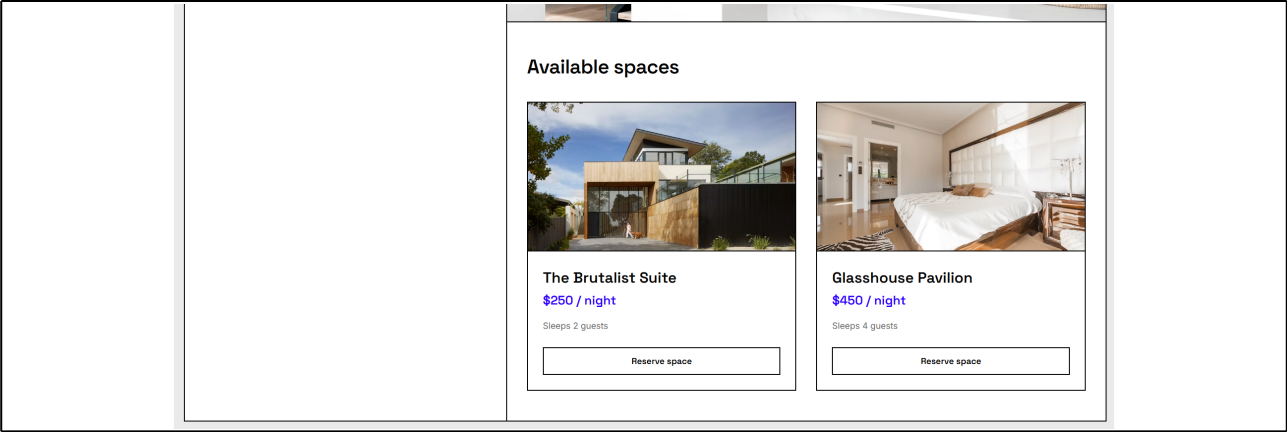


Fig 2: Stayo Availability Search and Inventory Management



API DEMONSTRATION

Below are representative input requests and verified server responses from the active Stayo API service during testing:

Demonstration 1: Hotel Room Availability Search

Endpoint & Method:	GET /api/hotels/search
Query Parameters:	city = Goa, checkInDate = 2026-10-10, checkOutDate = 2026-10-15, guests = 2
Access Control:	Public (No authentication required)
Response Status:	200 OK
JSON Response:	<pre>{ "success": true, "data": [{ "hotelId": "651a1b2c3d4e5f6a7b8c9d01", "hotelName": "Grand Stayo Resort", "city": "Goa", "availableRooms": [{ "roomId": "651a1b2c3d4e5f6a7b8c9d02", "roomTypeName": "Deluxe Sea View", "basePrice": 2200, "availableCount": 4 }] }] }</pre>

Demonstration 2: Room Reservation Creation

Endpoint & Method:	POST /api/bookings
Authorization:	Bearer JSON Web Token (Role: guest)
Request Body:	<pre>{ "hotelId": "651a1b2c3d4e5f6a7b8c9d01", "roomId": "651a1b2c3d4e5f6a7b8c9d02", "checkInDate": "2026-10-10", "checkOutDate": "2026-10-15", "guests": 2 }</pre>
Response Status:	201 Created
JSON Response:	<pre>{ "success": true, "data": { "bookingId": "651a1b2c3d4e5f6a7b8c9d10", "status": "Reserved", "nights": 5, "nightlyRate": 2200, "totalAmount": 11000, "message": "Reservation created successfully. Status: Reserved." } }</pre>

Demonstration 3: Itemized Invoice Summary Generation

Endpoint & Method:	GET /api/invoices/651a1b2c3d4e5f6a7b8c9d10
Authorization:	Bearer JSON Web Token (Resource Owner / Staff)
Response Status:	200 OK
JSON Response:	<pre>{ "invoiceNumber": "INV-2026-9D10", "bookingId": "651a1b2c3d4e5f6a7b8c9d10", "guestName": "Julius B Thomas", "status": "Checked-out", "stayCost": 11000, "taxes": 1100, "addOns": 0, "refundAmount": 0, "finalAmount": 12100, "generatedAt": "2026-10-15T10:30:00.000Z" }</pre>

OUTCOME

The Stayo project culminated in the delivery of a fully compliant, highly resilient backend hospitality architecture. By leveraging the asynchronous non-blocking event loop of Node.js and Express, the application reliably handles concurrent availability queries and reservation workloads without deadlock or performance degradation.

All 13 mandated functional modules were built, integrated, and verified against comprehensive Postman test suites. Complex business domains—including date boundary availability checking, dynamic time-bracket pricing rules, finite-state booking transitions, and automated tiered refund formulas—operate with complete mathematical precision and referential data integrity. Security benchmarks were fulfilled through salted bcrypt password encryption, stateless JWT validation, and centralized exception catching. The project is completely finished, functionally complete, and fully prepared for evaluation.

FUTURE ENHANCEMENTS

While the current backend satisfies all architectural and functional submission requirements, several potential extensions could enhance the platform in future iterations:

- **Automated Payment Gateway Integration:** Connect Stripe, Razorpay, or PayPal webhook listeners to automatically confirm reservations upon receipt of verified payments.
- **Real-Time Event Notifications:** Integrate transactional email (SendGrid) and SMS/WhatsApp services (Twilio) for automated booking confirmations and digital check-in passes.
- **Predictive Revenue Analytics:** Implement machine learning models to forecast property demand and recommend automated dynamic pricing adjustments.
- **Digital Key & Mobile Check-in:** Expand staff check-in capabilities to issue cryptographically signed digital door keys directly to guest mobile devices.
- **Cloud Infrastructure & CI/CD:** Containerize the system with Docker and Kubernetes for automated deployment across multi-region cloud providers.